

ECOSSISTEMA ÂNIMA / UNICURITIBA

Modelos, Métodos e Técnicas de Engenharia de Software
2026 – 1º Semestre

CALCULADORA DE PENA

Implementação de API no Sistema

Felipe Silva Muchinski
R.A.: 172422440

Curitiba
2026

SUMÁRIO

1 INTRODUÇÃO E ESCOPO DO SISTEMA.....	4
--	----------

1.1 Contexto do Projeto.....	4
1.2 Objetivos.....	4
1.3 Descrição do Escopo.....	4
2 LEVANTAMENTO DE REQUISITOS.....	4
2.1 Requisitos Funcionais (RF).....	4
2.2 Requisitos Não-Funcionais (RNF).....	7
3 MODELAGEM E DIAGRAMAS DO SISTEMA.....	8
3.1 Diagrama de Classes.....	9
3.2 Outros Diagramas de Engenharia de Software.....	11
4 ARQUITETURA DO SISTEMA.....	12
4.1 Visão Geral da Arquitetura.....	12
4.2 Tecnologias Utilizadas.....	13
5 MODELAGEM E BANCO DE DADOS.....	16
5.1 Modelo do Banco de Dados.....	16
5.2 Tecnologia Escolhida.....	16
6 DIVISÃO DE TAREFAS.....	20
7 CONCLUSÃO E REFERÊNCIAS.....	20
7.1 Conclusão.....	20
7.2 Referências Bibliográficas.....	21

1 INTRODUÇÃO E ESCOPO DO SISTEMA

1.1 Contexto do Projeto

Uma ferramenta capaz de facilitar a atualização das leis para facilitar o cálculo de penas de réus para atuantes da área jurídica penal.

1.2 Objetivos

Ajudar a atualizar as leis usadas pelo sistema conforme as atualizações do Código Penal Brasileiro, realizando os cálculos penais com mais agilidade e versatilidade.

1.3 Descrição do Escopo

A API realiza a ponte entre o sistema de cálculo e o banco de dados com as leis que efetuam o ajuste de pena máxima e mínima conforme os respectivos agravantes, atenuantes, circunstâncias judiciais e causas de aumento e diminuição.

2 LEVANTAMENTO DE REQUISITOS

2.1 Requisitos Funcionais (RF)

Login

ID	Nome	Descrição	Prioridade
RF-01	Login com credenciais	O sistema deve permitir acesso mediante e-mail e senha cadastrados no dicionário USERS de app.py	Alta
RF-02	Redirecionamento não autenticado	Qualquer rota da calculadora deve redirecionar para /login quando não há sessão ativa	Alta
RF-03	Logout	O usuário deve poder encerrar a sessão; a sessão é limpa e o usuário redirecionado para /login	Alta
RF-04	Feedback de autenticação	Exibir mensagens flash de sucesso/erro após tentativa de login ou logout	Média

Etapa 1

ID	Nome	Descrição	Prioridade
RF-05	Dados do processo	O usuário deve informar nome do réu e número do processo (campos opcionais)	Média
RF-06	Pena em abstrato	O usuário deve informar os limites mínimo e máximo da pena prevista no tipo penal (em anos e meses)	Alta
RF-07	Validação dos limites	O sistema deve rejeitar cálculos quando $\text{min} \leq 0$, $\text{max} \leq 0$ ou $\text{min} > \text{max}$, retornando <code>is_valid=False</code>	Alta
RF-08	Stepper numérico	Botões + e - devem ajustar anos (0–99) e meses (0–11) com atualização imediata do painel	Média

Etapa 2

ID	Nome	Descrição	Prioridade
RF-09	Fase 1 — Circunstâncias judiciais	O sistema deve aplicar até 8 circunstâncias judiciais negativas (Art. 59), deslocando a pena-base proporcionalmente entre o mínimo e o máximo	Alta
RF-10	Fase 2 — Agravantes (Art. 61)	Cada agravante selecionado acrescenta 1/6 sobre a pena-base (9 agravantes disponíveis)	Alta
RF-11	Fase 2 — Atenuantes (Arts. 65 e 66)	Cada atenuante selecionado deduz 1/6 da pena-base; inclui atenuante nominada do Art. 66 (9 atenuantes disponíveis)	Alta
RF-12	Piso e teto da Fase 2	A pena intermediária não pode ficar abaixo do mínimo em abstrato nem acima de 1,5× o máximo	Alta
RF-13	Fase 3 — Causas de aumento	Aplicar até 6 causas de aumento: frações normais (1/6 a 2/3), aumento fixo de +2/3 (arma uso permitido) e dobro (arma uso restrito)	Alta
RF-14	Fase 3 — Causas de diminuição	Aplicar até 5 causas de diminuição: Tentativa, Participação de menor importância, Colaboração premiada, Arrependimento Posterior (Art. 16) e Tráfico Privilegiado (Art. 33 §4º)	Alta
RF-15	Aumento fixo de +2/3 — Arma uso permitido	Ao selecionar esta opção no Art. 157 §2-A I, a fase 3 multiplica a pena por 5/3 (não é fração variável)	Alta
RF-16	Aumento em dobro — Arma uso restrito	Ao selecionar esta opção no Art. 157 §2-A II, a fase 3 multiplica a pena por 2 (dobro)	Alta
RF-17	Painel em tempo real	O painel direito deve atualizar anos/meses/dias e as três fases via AJAX a cada alteração no formulário, sem recarregar a página	Alta

Etapa 3

ID	Nome	Descrição	Prioridade
RF-18	Exibição das três fases	Mostrar pena-base, intermediária e definitiva com o total em dias e a formatação por extenso	Alta
RF-19	Sugestão de regime	Com base na pena definitiva, sugerir regime Aberto (< 4 anos), Semiaberto (4–7 anos) ou Fechado (≥ 8 anos) com referência ao artigo	Alta
RF-20	Impressão / PDF	O botão Imprimir deve acionar window.print(); o CSS de impressão oculta sidebar, painel e botões	Média

RF-21	Novo cálculo	O botão 'Novo Cálculo' deve limpar o estado da sessão e retornar à Etapa 1	Média
RF-22	Voltar para Etapa 2	O botão 'Voltar' deve renderizar a Etapa 2 mantendo todos os valores previamente selecionados	Média

Módulo de API (JSON)

ID	Nome	Descrição	Prioridade
RF-23	Endpoint POST /calculate	Receber JSON com o estado completo e retornar o resultado das 3 fases em JSON (usado pelo painel em tempo real)	Alta
RF-24	Proteção da API	O endpoint /calculate deve retornar HTTP 401 quando chamado sem sessão autenticada	Alta

2.2 Requisitos Não-Funcionais (RNF)

Desempenho

ID	Nome	Descrição
RNF-01	Resposta da API	O endpoint /calculate deve responder em menos de 200ms em hardware local (o cálculo é puramente matemático, sem I/O)
RNF-02	Atualização do painel	O painel em tempo real deve refletir alterações em menos de 500ms incluindo latência de rede local
RNF-03	Renderização inicial	Cada página deve carregar em menos de 1 segundo em conexão local

Segurança

ID	Nome	Descrição
RNF-04	Autenticação obrigatória	Todas as rotas da calculadora exigem sessão ativa; redirecionam para /login caso contrário
RNF-05	Chave de sessão segura	A secret_key do Flask deve ser substituída por valor aleatório longo antes de implantação em produção

Usabilidade

ID	Nome	Descrição
RNF-06	Feedback visual imediato	Checkboxes, toggles e seletores de fração devem alterar visualmente o estado do card ao serem ativados
RNF-07	Painel lateral persistente	O painel direito com a pena calculada deve estar sempre visível durante as Etapas 1 e 2
RNF-08	Indicador de progresso	A barra de progresso no painel deve mostrar onde a pena definitiva está no intervalo entre mínimo e máximo
RNF-09	Acordeões expansíveis	As três fases da Etapa 2 devem ser exibidas em acordeões que podem ser expandidos/recolhidos independentemente

Manutenibilidade

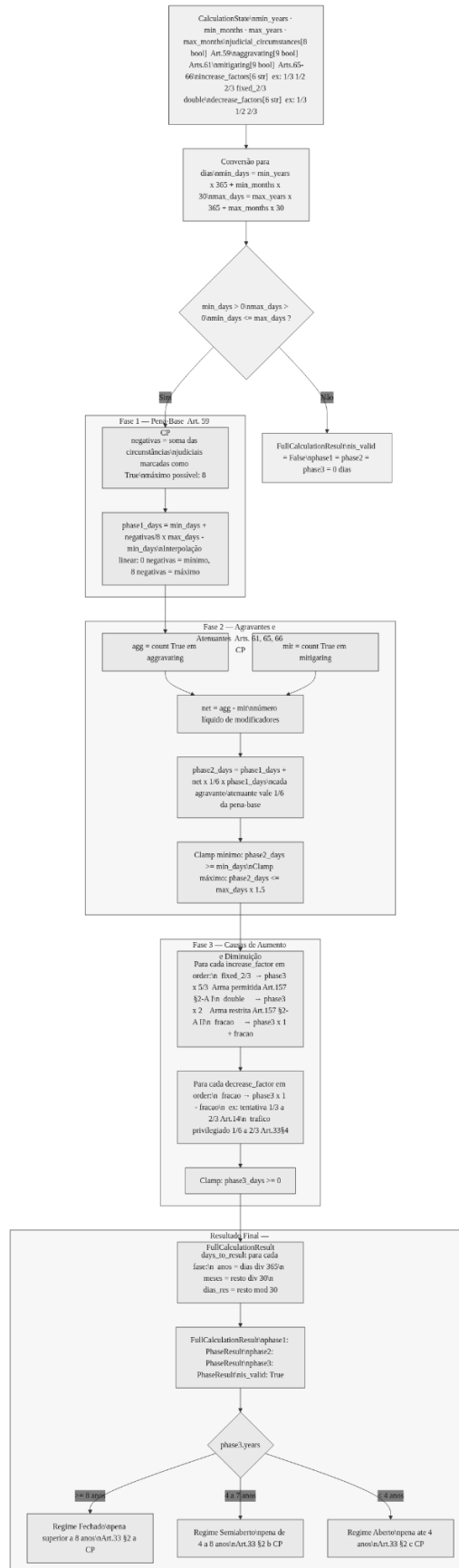
ID	Nome	Descrição
RNF-10	Separação lógica	A lógica de cálculo deve estar isolada em calculation.py, sem dependências de Flask ou HTML
RNF-11	Dados configuráveis	Listas de circunstâncias, agravantes, atenuantes e causas devem estar em dicionários Python em app.py, fáceis de editar
RNF-12	Paleta via variáveis CSS	Todas as cores do site devem ser controladas por variáveis CSS no bloco :root de base.html, sem cores hard-coded
RNF-13	Testabilidade	O módulo calculation.py deve poder ser importado e testado com pytest sem necessidade de iniciar o servidor Flask

Portabilidade e Implantação

ID	Nome	Descrição
RNF-14	Dependências mínimas	O sistema requer apenas Flask (pip install flask); sem banco de dados, sem bibliotecas de terceiros adicionais
RNF-15	Compatibilidade Python	Compatível com Python 3.10+; usa dataclasses e type hints modernos
RNF-16	Navegadores suportados	Compatível com Chrome, Firefox, Edge e Safari modernos; usa apenas CSS variables e Fetch API padrão

3 MODELAGEM E DIAGRAMAS DO SISTEMA

3.1 Diagrama de Classes



3.1.1 Processamento e Cálculo de Pena

Preparação: o sistema recebe os limites da pena e marcadores (agravantes etc.), converte tudo para dias e valida a consistência dos dados.

Fase 1 – Pena-Base: define o ponto de partida entre o mínimo e o máximo via interpolação linear das 8 circunstâncias judiciais (Art. 59).

Fase 2 – Intermediária: aplica agravantes (+1/6) e atenuantes (-1/6), respeitando os limites legais.

Fase 3 – Causas de Aumento/Diminuição: aplica multiplicadores finais (ex.: tentativa, armas, tráfico privilegiado).

3.1.2 Resultado e Regime

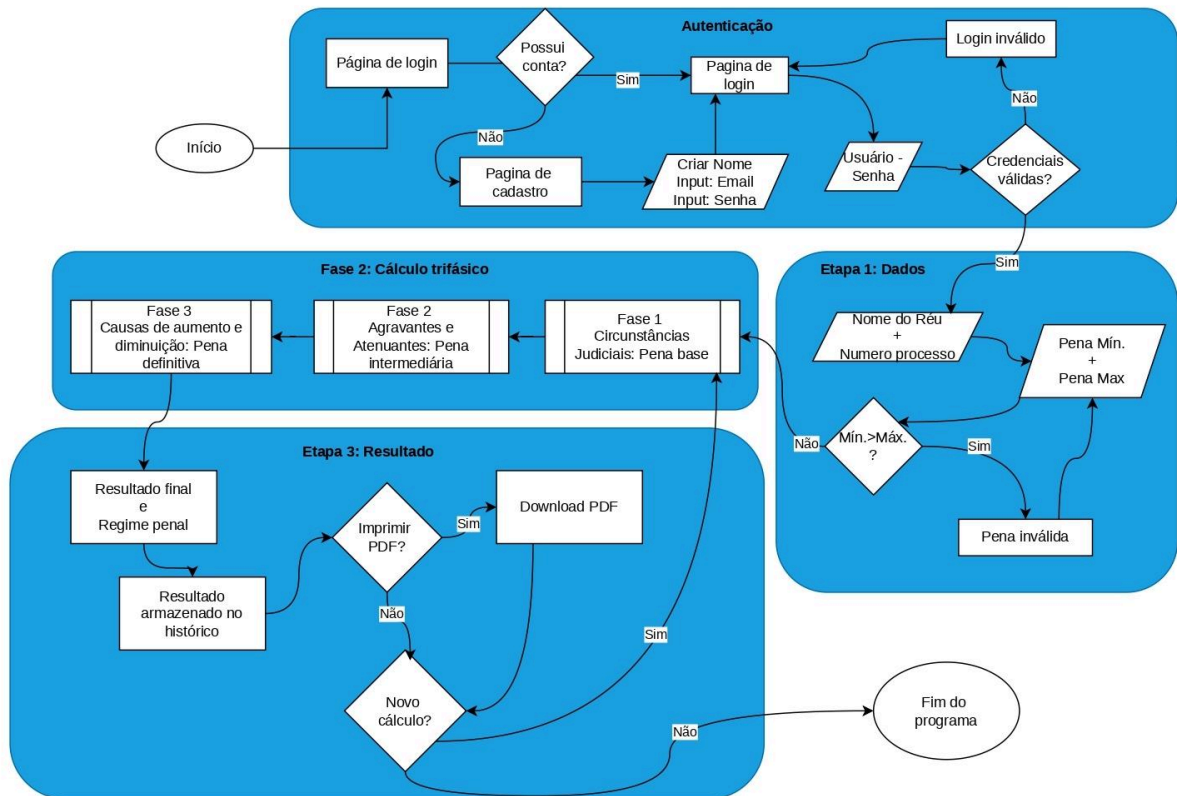
A pena é reconvertida para anos/meses/dias, definindo o regime prisional: Fechado (> 8 anos); Semiaberto (4 a 8 anos); Aberto (< 4 anos).

Resumo: a lógica traduz o critério trifásico do Código Penal em um fluxo matemático automatizado para o cálculo da sentença.

3.2 Outros Diagramas de Engenharia de Software

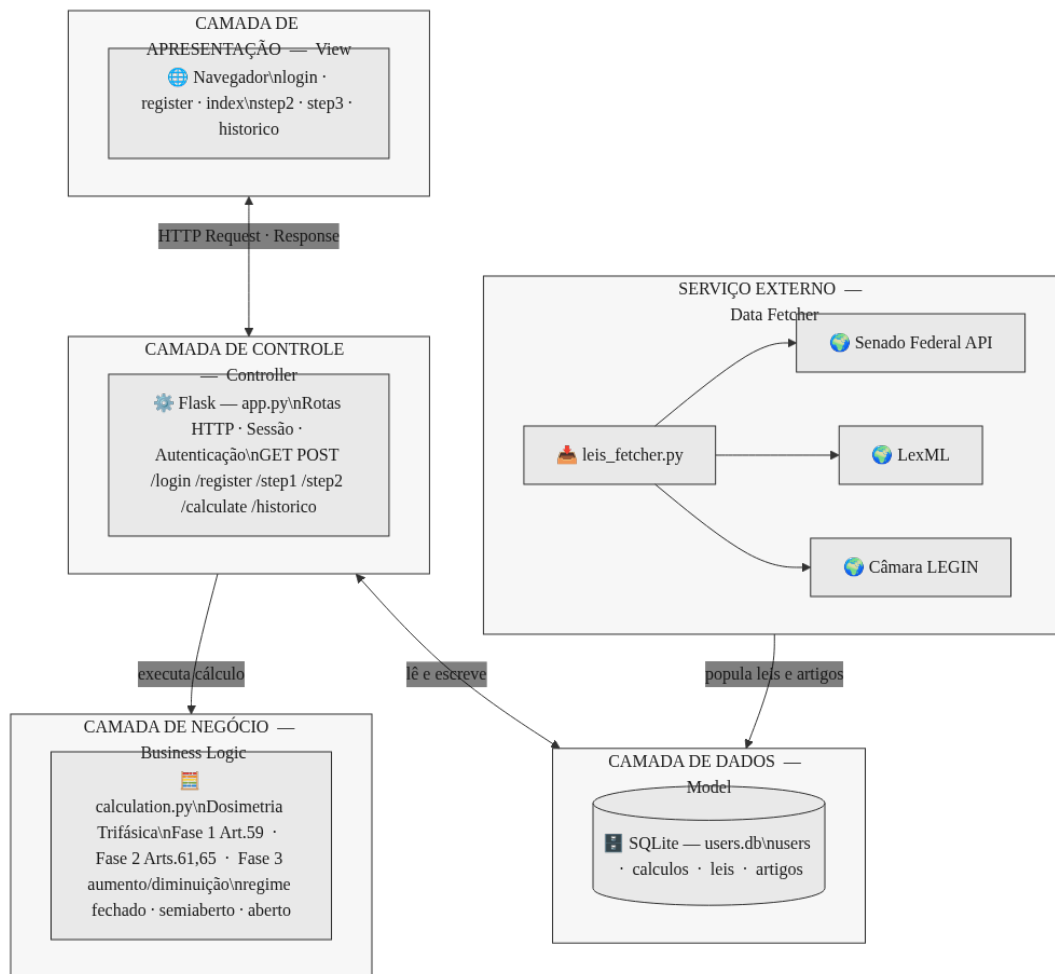
Jornada do Usuário

Este diagrama detalha o fluxo operacional do sistema jurídico, abrangendo desde a autenticação do usuário e a coleta de dados do réu até a execução do cálculo trifásico da pena e a geração do resultado final.



4 ARQUITETURA DO SISTEMA

4.1 Visão Geral da Arquitetura



Trata-se de um Diagrama Entidade-Relacionamento (ER), que descreve a arquitetura das tabelas e os vínculos lógicos do banco de dados.

4.1.1 Estrutura das Tabelas

users (Usuários): ID, nome, e-mail (chave única) e senha criptografada (hash).

calculos (Cálculos): histórico de penas calculadas, registra dados do réu, processo, resultados das três fases e regime final; vincula-se ao usuário pelo e-mail.

leis (Legislação): acervo jurídico (ex.: Código Penal), com ementa, conteúdo em HTML e links para fontes oficiais (URN/Legin).

artigos (Artigos): fragmentos individuais das leis; cada registro contém o número e o texto do artigo, vinculado a uma lei específica.

4.1.2 Relacionamentos Principais

Usuário → Cálculos: um usuário pode realizar e salvar múltiplos cálculos (1:N).

Lei → Artigos: uma lei é composta por diversos artigos (1:N).

O diagrama organiza o ecossistema do sistema, conectando quem utiliza a ferramenta (usuários), o que é produzido (cálculos) e a base legal consultada (leis e artigos).

4.2 Tecnologias Utilizadas

4.2.1 Linguagem

Python 3.11+ — linguagem principal do back-end e scripts de dados.

4.2.2 Framework Web

Flask — micro-framework web para Python; gerencia rotas HTTP, sessões, templates e flash messages.

4.2.3 Bibliotecas Python

Biblioteca	Uso
sqlite3	Conexão e operações com o banco de dados (built-in do Python)
werkzeug.security	Hash de senha com pbkdf2:sha256 para autenticação segura
requests	Requisições HTTP para as APIs externas (Senado, LexML, Câmara)
urllib3	Desativa avisos de SSL para APIs governamentais com certificado autoassinado
re	Expressões regulares para extração de artigos do HTML das leis
dataclasses	Define CalculationState, PhaseResult, FullCalculationResult
datetime	Data atual exibida no resultado final
time	sleep(0.5) entre chamadas às APIs para não sobrecarregá-las
os	Resolve o caminho absoluto do banco de dados (DB_PATH)

4.2.4 Front-end

HTML5 + CSS3: estrutura e estilo de todas as páginas.

Jinja2: motor de templates do Flask; renderiza variáveis Python no HTML.

JavaScript (Vanilla): live preview da pena em tempo real via fetch() API sem recarregar a página.

Google Fonts (Inter): tipografia via CDN.

4.2.5 Banco de Dados

SQLite: banco de dados relacional em arquivo único (users.db), sem servidor separado.

WAL Mode (PRAGMA journal_mode=WAL): melhora a performance em leituras concorrentes.

4.2.6 APIs Externas

API	URL	Uso
Senado Federal	legis.senado.leg.br/dadosabertos	Busca metadados da lei (nome, ementa, URN)
LexML	lexml.gov.br	Resolve o URN para o link da Câmara
Câmara LEGIN	camara.leg.br/legin	Baixa o texto compilado e atualizado da lei

4.2.7 Ferramentas de Desenvolvimento

Git — controle de versão.

mermaid.live — geração dos diagramas UML.

VS Code — IDE.

4.2.8 Padrão Arquitetural

Arquitetura em Camadas (inspirada em MVC): Visão (templates), Controle (rotas Flask), Lógica de Negócios (calculation.py) e Modelo (SQLite).

Cache-first pattern: leis_fetcher.py sempre consulta o banco antes de chamar a API externa.

5 MODELAGEM E BANCO DE DADOS

5.1 Modelo do Banco de Dados

users		
INTEGER	id	PK
TEXT	name	
TEXT	email	UK
TEXT	password_hash	

salva

calculos		
INTEGER	id	PK
TEXT	user_email	FK
TEXT	reu_name	
TEXT	process_number	
INTEGER	fase1_dias	
INTEGER	fase2_dias	
INTEGER	fase3_anos	
INTEGER	fase3_meses	
INTEGER	fase3_dias_res	
INTEGER	fase3_total	
TEXT	regime	
TEXT	salvo_em	

leis		
INTEGER	id	PK
TEXT	tipo	
TEXT	numero	UK
TEXT	nome	
TEXT	ementa	
TEXT	urn	
TEXT	login_link	
TEXT	compiled_html	
TEXT	fetched_at	

contém

artigos		
INTEGER	id	PK
INTEGER	lei_id	FK
INTEGER	artigo_numero	
TEXT	texto	
TEXT	fetched_at	

5.2 Tecnologia Escolhida

5.2.1 Explicação da Arquitetura — Pena Calculada

Este projeto usa uma Arquitetura em Camadas (Layered Architecture), inspirada no padrão MVC (Model-View-Controller). O importante é separar o sistema em partes com suas devidas responsabilidades, concedendo uma visão e entendimento mais claro da arquitetura ao ser analisada e executada.

5.2.2 As Camadas do Sistema

1. Visão (HTML + Jinja2 + JavaScript)

Os arquivos na pasta templates/ são o que aparece no navegador do usuário. Eles não sabem nada sobre banco de dados ou cálculos — só exibem informações.

O Flask usa Jinja2 como motor de templates, o que permite misturar Python com HTML de forma segura:

```
<!-- Jinja2 renderiza a variável Python direto no HTML -->
<span>Réu: {{ reu_name }}</span>
```

Sem Jinja2, teria que montar o HTML manualmente como string no Python, algo que fica muito propenso a erros de segurança.

Para um projeto acadêmico e de escopo definido, optou-se por não utilizar frameworks como React, que trariam complexidade desnecessária. O JavaScript tem uma única função: atualizar o painel de pena em tempo real quando o usuário muda um valor, sem recarregar a página inteira.

Isso é feito com `fetch()`, uma função nativa do navegador:

```
fetch('/calculate', { method: 'POST', body: JSON.stringify(dados) })
  .then(r => r.json())
  .then(data => updatePanel(data));
```

Resultado: experiência fluida sem a complexidade de um front-end de framework.

2. Controle (Flask — app.py)

O Flask é o "gerente" da aplicação. Quando o usuário clica em um botão ou acessa uma URL, o Flask decide o que fazer.

Flask foi escolhido por ser minimalista: ele faz só o roteamento e deixa o desenvolvedor escolher o restante. Isso significa mais código manual, mas também mais aprendizado real:

```
@app.route("/login", methods=["GET", "POST"])
def entrar():
    usuario = conn.execute("SELECT * FROM users WHERE email=?",
        (email,)).fetchone()
    if check_password_hash(usuario["password_hash"], senha):
        session["user_email"] = email
```

As sessões do Flask resolvem a persistência do estado de autenticação, guardando dados do usuário em um cookie criptografado. Assim, o sistema sabe que o usuário está logado enquanto navega entre as páginas.

3. Lógica de Negócios (calculation.py)

Este é o módulo mais importante do ponto de vista jurídico, contendo toda a lógica do método trifásico de dosimetria penal (Arts. 59, 61 e 65 do Código Penal).

Caso o cálculo estivesse dentro de `app.py`, para testá-lo ou corrigi-lo seria necessário entender Flask, banco de dados e cálculo ao mesmo tempo. Separado, é possível importar e testar `calculate()` de forma completamente independente:

```
from calculation import CalculationState, calculate
estado = CalculationState(min_years=4, max_years=8)
resultado = calculate(estado)
print(resultado.phase1.days) # funciona sem Flask, sem banco
```

Em vez de passar dicionários soltos, utilizou-se `@dataclass` para definir estruturas com campos nomeados e valores padrão:

```
@dataclass
class CalculationState:
    min_years: int = 0
    max_years: int = 0
    judicial_circumstances: List[bool] = field(default_factory=lambda:
[False] * 8)
```

Isso garante que sempre é possível saber exatamente quais dados o cálculo precisa.

4. Modelo (SQLite — users.db)

O banco de dados guarda tudo que precisa persistir entre sessões: usuários, cálculos salvos, leis e artigos.

SQLite é um banco de dados que vive em um único arquivo (.db), não precisa de servidor separado, configuração ou credenciais, sendo a escolha ideal para o projeto:

```
conn = sqlite3.connect("users.db") # abre (ou cria) o banco em uma
linha
```

Cada tabela representa uma entidade diferente do sistema, seguindo os princípios de normalização para evitar dados duplicados e inconsistentes:

Tabela	O que guarda
users	Dados de autenticação
calculos	Resultados dos cálculos por usuário
leis	Metadados das leis federais
artigos	Texto de cada artigo das leis

5. Serviço Externo (leis_fetcher.py)

O módulo `leis_fetcher.py` gerencia a ingestão de dados governamentais. A busca de leis é realizada de forma não simultânea ao fluxo principal do usuário, protegendo a aplicação contra instabilidades nas APIs do Senado ou da Câmara.

Padrão Cache-first: o sistema prioriza a consulta ao banco de dados local antes de realizar chamadas externas, otimizando o tempo de resposta e respeitando os limites de requisição (rate limiting) dos provedores oficiais. Isso evita chamadas repetidas à API,

respeita os limites de requisição do governo e faz a aplicação responder instantaneamente para dados já coletados.

5.2.3 Resumo da Escolha Arquitetural

Decisão	Alternativa Descartada	Motivo da Escolha
Flask	Django	Mais simples para aprender; sem "mágica" escondida
SQLite	PostgreSQL / MySQL	Sem servidor; zero configuração; ideal para MVP
Jinja2 + HTML	React / Vue	Sem build tools; mais direto para quem está aprendendo
JavaScript Vanilla	jQuery / Axios	Nativo do navegador; sem dependências extras
Módulos separados	Tudo em app.py	Separação de responsabilidades; mais fácil de testar e manter
Cache-first	Chamar API sempre	Performance; resiliência a falhas externas

6 DIVISÃO DE TAREFAS

[Conteúdo a ser preenchido conforme original]

7 CONCLUSÃO E REFERÊNCIAS

7.1 Conclusão

7.1.2 Desafios Enfrentados

Complexidade Regulatória: traduzir a lógica jurídica para convertê-la em números e informações intuitivas na hora do manuseio da interface.

Arquitetura e Integração: configurar o ambiente de desenvolvimento no VS Code integrando o Flask com bancos de dados e tabelas gerenciadas e garantir a persistência correta dos históricos de cálculo.

Design de Interface (UI/UX): criar uma interface intuitiva que facilitasse a inserção de dados técnicos (como penas mínimas e máximas) sem sobrecarregar o usuário com informações a serem interpretadas.

7.1.3 Lições Aprendidas

Foco em rigor técnico: a etapa inicial deste projeto representou o desafio mais significativo, dada a complexidade da interseção entre o Direito e a Tecnologia. A transposição de normas jurídicas para algoritmos matemáticos exige um elevado nível de responsabilidade e precisão interpretativa, garantindo que a lógica normativa seja convertida fielmente em parâmetros computacionais.

Validação de Dados: os diagramas evidenciam a importância de etapas de verificação, como a checagem se a pena mínima é superior à máxima, para evitar erros em cascata no resultado final.

Gerenciamento de Projetos Acadêmicos: a experiência de conciliar o desenvolvimento deste SaaS com as demandas de duas matérias de Ciência da Computação e projetos de extensão reforçou a importância do cumprimento de prazos e da organização técnica de projetos e estudos durante e fora do período de aulas.

7.2 Referências Bibliográficas

LEXML. LexML Brasil. Disponível em: <https://www.lexml.gov.br/>. Acesso em: maio 2026.

SENADO FEDERAL. Atividade – Comissões. Disponível em: <https://legis.senado.leg.br/atividade/comissoes/>. Acesso em: maio 2026.

DRAWIO. Diagrams.net. Disponível em: <https://www.drawio.com/>. Acesso em: maio 2026.

MERMAID. Mermaid – Diagramação e Gráficos para Markdown. Disponível em: <https://mermaid.js.org/>. Acesso em: maio 2026.

GOOGLE. Gemini. Disponível em: <https://gemini.google.com/app?hl=pt-BR>. Acesso em: maio 2026.

ANTHROPIC. Claude AI. Disponível em: <https://claude.ai/>. Acesso em: maio 2026.

OVERLEAF. Overleaf – Editor LaTeX Online. Disponível em: <https://pt.overleaf.com/>. Acesso em: maio 2026.